"LLM의 실시간 정보 탐색과 지식 그래프 아카이빙을 결합한 지식 관리 서비스"

아이디어 개요

KnowledgeArc(Knowledge + Archive 가칭)는 휘발되기 쉬운 AI 와의 대화를 내 지식으로 영구 저장하는 'RAG 기반 올인원 학습 플랫폼'입니다.

검색부터 정리까지 한 화면에서 해결하며, 쌓인 지식은 AI 가 자동으로 연결하고 시각화하여 나만의 **'두 번째 뇌(Second Brain)'를 구축해 줍니다.

1. 아이디어 선정 이유 (문제점 및 개선점)

- 기존의 문제점 (Pain Point): "지식의 휘발성과 파편화"
  - **휘발성:** ChatGPT 나 Gemini 에게 유용한 정보를 얻어도, 채팅방이 닫히거나 대화가 길어지면 그 지식은 로그 속에 묻혀버립니다.
  - **파편화:** AI 에게 얻은 정보를 별도로 노션(Notion)이나 옵시디언(Obsidian)으로 옮기려면 '복사+붙여넣기'라는 귀찮은 과정이 필요합니다. 이 과정에서 맥락(Context)이 끊기고, 나중에 다시 찾으려 해도 "그때 뭐라고 검색했더라?" 하며 헤매게 됩니다.
- 개선점 (Solution): "학습 흐름의 끊김 없는 연결(Seamless Flow)"
  - One-Stop: 검색(질문)과 동시에 문서화가 이루어집니다.
  - **자동 구조화:** AI 가 답변을 주는 것에서 끝나지 않고, 해당 내용의 **핵심 키워드를 추출하여 자동으로 태그를 달고 분류**해 줍니다.
  - **맥락 보존:** 과거에 내가 학습했던 내용을 AI 가 기억하고, 새로운 질문을 할 때 과거의 내 기록을 참고하여 답변해 줍니다 (RAG 기술 활용).

---

2. BE (백엔드) 관점: 무엇을 만들고 해결해야 할까?

백엔드는 "데이터의 저장"을 넘어 "데이터의 의미 파악 및 검색"에 집중해야 합니다.

핵심 기능:

1. **LLM API 연동 및 스트리밍:** OpenAI, Anthropic 등의 API 를 호출하고, 답변을 실시간(SSE, Server-Sent Events)으로 프런트엔드에 전달하는 파이프라인 구축.

2. **벡터 검색 (Vector Search) 구현:**
   - 사용자의 저장된 메모를 AI 가 이해할 수 있는 형태(Vector Embedding)로 변환하여 저장해야 합니다.
   - 기존 RDBMS(MySQL) 외에 **Vector DB**(Pinecone, Milvus) 또는 MySQL 의 벡터 기능 등을 활용해 '의미 기반 검색'을 구현해야 합니다.

3. **자동 분류 파이프라인:**
   - 사용자가 글을 저장할 때, 백그라운드에서 AI 를 한 번 더 호출하여 "이 글의 카테고리는 무엇이고, 태그는 #Java #Spring 이다"라고 메타데이터를 생성해 주는 로직.

- **해결해야 할 기술적 챌린지:**
  - **토큰 관리:** 긴 문서를 저장할 때 LLM 의 토큰 제한을 어떻게 처리할 것인가? (Text Splitter 구현)
  - **RAG (검색 증강 생성) 정확도:** 사용자가 "지난번에 공부한 거 요약해줘"라고 했을 때, 정확히 관련 문서를 찾아내는 알고리즘.

---

3. FE (프런트엔드) 관점: 무엇을 만들고 해결해야 할까?

프런트엔드는 "쓰기 경험(Editor)"과 "지식의 시각화(Graph)"가 핵심입니다.

핵심 기능:

1. **WYSIWYG 에디터 고도화:**
   - 단순 텍스트 박스가 아니라, Notion 처럼 블록 단위로 AI 답변을 수정하고 내 생각을 덧붙일 수 있는 에디터

     WYSIWYG 는 **"What You See Is What You Get"**의 약자로, 읽을 때는 **"위지윅"**이라고 부릅니다.

     한국어로 직역하면 **"보는 대로 얻는다"**는 뜻입니다. 문서나 이미지를 편집할 때, **화면에서 보이는 모습 그대로 최종 결과물(출력물, 웹페이지 등)이 나오는 방식**을 말합니다.

     차이점 비교 (HTML vs WYSIWYG)

     만약 **"안녕하세요"**라는 글자를 **굵게(Bold)** 만들고 싶다면?

- **HTML/코드 방식:**
- 입력: 안녕하세요 또는 *안녕하세요**
- 화면: 코드가 보임. 렌더링 버튼을 눌러야 굵은 글씨 확인 가능.

- **WYSIWYG (위지윅) 방식:**
- 입력: '안녕하세요'를 드래그하고 '굵게' 버튼 클릭
- 화면: **안녕하세요** (그 자리에서 바로 굵게 변함)

이미 익숙하게 쓰고 계신 툴들이 대부분 위지윅 방식입니다.

- **MS Word, 한글:** 작성하는 화면 그대로 종이에 인쇄됩니다.
- **Notion (노션):** /를 눌러 헤딩을 선택하면 글자가 바로 커집니다.
- **블로그 에디터:** 네이버 블로그나 티스토리의 글쓰기 화면도 위지윅입니다.

2. **지식 그래프 시각화 (Obsidian-like):**
- 문서 간의 연결 관계(Link)를 시각적으로 보여주는 그래프 뷰 구현
- A 문서에서 B 문서를 언급하면 선으로 연결되는 UI.

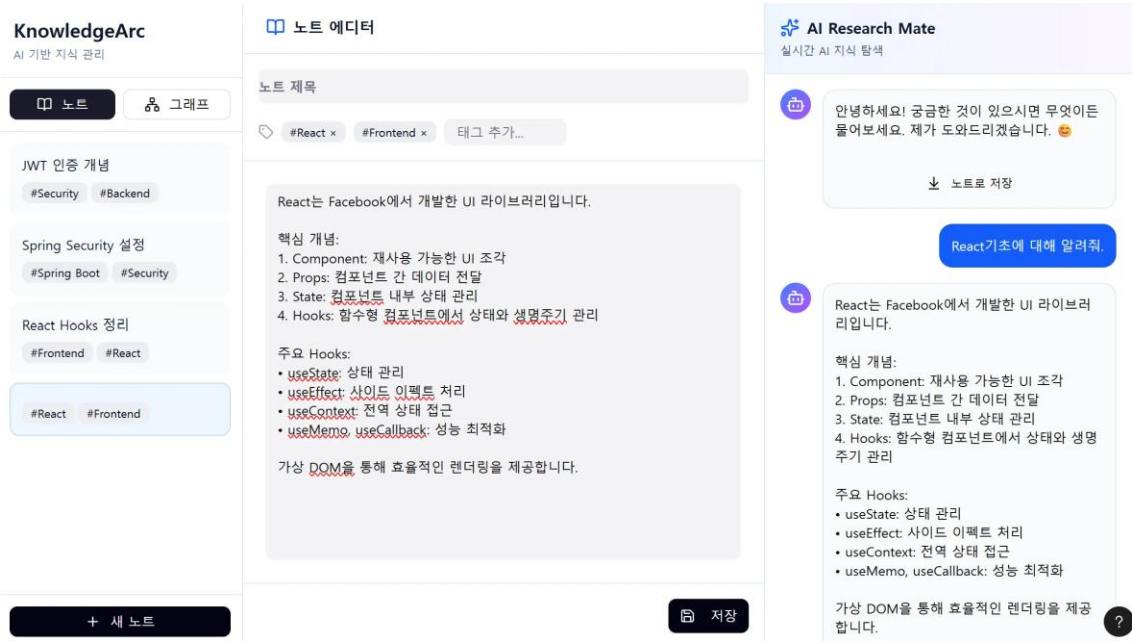
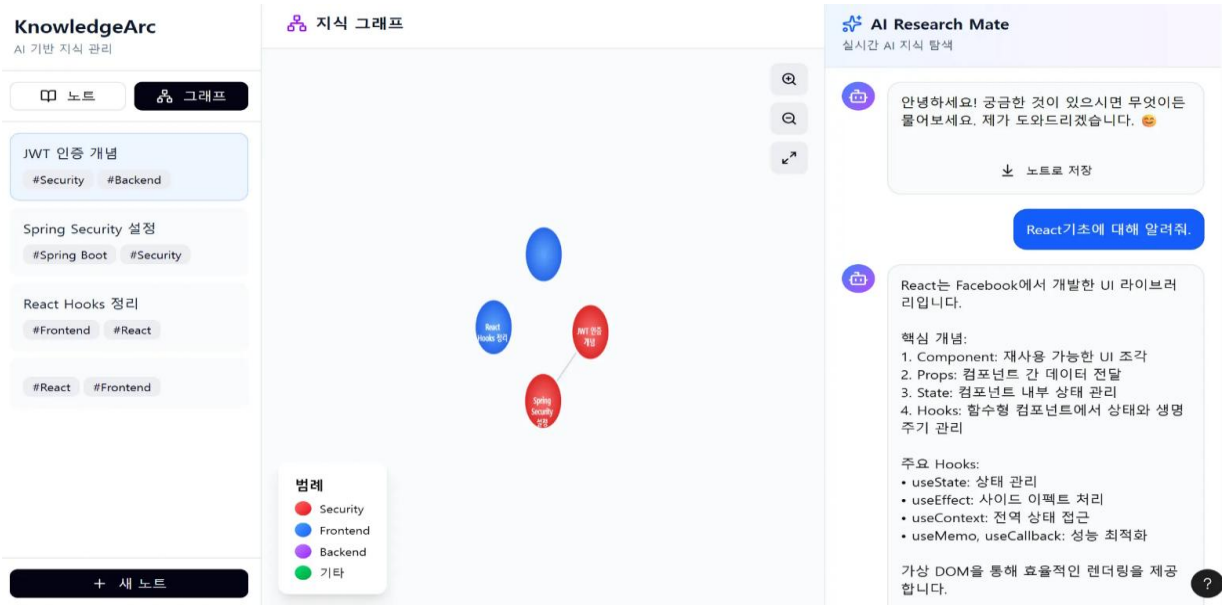
3. **사이드바의 맥락 추천 UI:**
- 글을 쓰고 있을 때, 우측 사이드바에 "관련된 과거 메모"를 자동으로 띄워주는 UX.

- **해결해야 할 기술적 챌린지:**
  1. **마크다운(Markdown) 렌더링 최적화:** AI 가 보내주는 마크다운 텍스트를 실시간으로 깔끔한 UI 로 변환하면서, 동시에 사용자가 편집 가능하게 만드는 것.
  2. **상태 관리 복잡도:** 현재 편집 중인 문서, AI 와의 채팅 상태, 그래프 데이터 간의 동기화 문제 해결 .

---

4. 이 아이디어를 통해 얻을 수 있는 효과

- **사용자 입장 (User Benefit):**
  - **나만의 위키 구축:** 별도의 정리 시간을 들이지 않아도, 공부하면 할수록 자동으로 나만의 지식 백과사전이 만들어집니다.

    - **생산성 향상:** "찾고 -> 읽고 -> 정리하는" 3 단계를 1 단계로 압축하여 학습 효율이 극대화됩니다.
- 개발자 입장 (Portfolio Effect):
    - **최신 트렌드 기술 증명:** 단순 CRUD 게시판이 아니라, **LLM 연동, 벡터 DB, RAG, 복잡한 에디터 구현** 등 현업에서 가장 주목하는 기술 역량을 보여줄 수 있습니다.
    - **풀스택 경험:** 복잡한 프런트엔드 인터렉션(그래프, 에디터)과 백엔드의 데이터 처리 로직을 모두 깊이 있게 다룰 수 있습니다.

Figma 로 생성한 예시 이미지